Software Design Document (SDD)
Assignment #2 Assembler for XE computer
CS530, Spring 2026

Team:

Jorge Arellano, cssc2506, Red ID: 131416399

Manuel Jimenez, cssc2526, Red ID: 828131233

Overview & Goals:
This project implements a limited SIC/XE two-pass assembler program variant that assembles code from a SIC/XE source file (.sic). The assembler will support most of SIC/XE instructions and directives, except EQU and ORG directives. It will output a Symtab file(.st) and a listing file (.l) for each input source file.

Goals:

1. Open SIC/XE source file (P2sample.sic) and generate listing files for the XE machine using instructions and directive.
2. Develop an algorithm for a limited two-pass assembler .
3. Using said algorithm, create a Symtab file and listing file from the given source SIC/XE file

Project Description:
The purpose of this project is to create a limited SIC/XE assembler that will open a given SIC/XE source file. This project will be able to use the SIC/XE op table to translate the opcode and print out the right operations, along with the registers/labels that are used from the symbol table. In pass 1, the assembler parses source file, build SYMTAB, and track memory address using LOCCtR. In pass 2 uses opcode table and symbol table to generate listing file and validate symbol and opcode usage. The output will then produce a symbol file(.st) and listing file(.l). There will be error catches and message if the file is not correct and the inputs are not valid. If the symbol file and object file are not present, the program will then exit.

Plan of Action and Milestones (POAM):
Phase One: Outline/ Brainstorm-(Mar 3rd-Mar 8th)

- Identify the final goals for the project and software implementation.
- Discuss timeframe and team roles/ workload distribution.
- Question what problems may arise during implementation and discuss techniques to remedy any issues.

Phase Two: Create Pseudocode-(Mar 13th- Mar 18th)

- Create a Data Flow Diagram.
- Design Psuedocode and establish absolute requirements
- Coding roles to be determined after the Pseudocode phase. Allows for logical workloads to be assigned to each individual programmer.

•Address the concerns regarding the desired functionality from each programmer.

Phase Three: Initial Stage Coding-(Mar 19th-Mar 25th)

- Implement source code from Pseudocode.
- Build Pass 1 maintain Location counter (LOCCTR) and Pass 2
- Add Test cases for ensuring functionality
- Prepare for Final Stage Coding.

Phase Four: Final Stage Coding-(Mar 26th-Apr 9th)

- Finalize Initial coding by debugging any errors.
- Add on different features that need to be implemented.
- Pinpoint any errors or shortcomings from coding.
- Combine both Stages of coding.

Phase Five: Finalization-(Apr 10th–Apr 21st)

- Complete all debugging and error finding of previous phase.
- Finishing adding all requirements and features for program.
- Testing to ensure flawless function.
- Final submission April 21st.

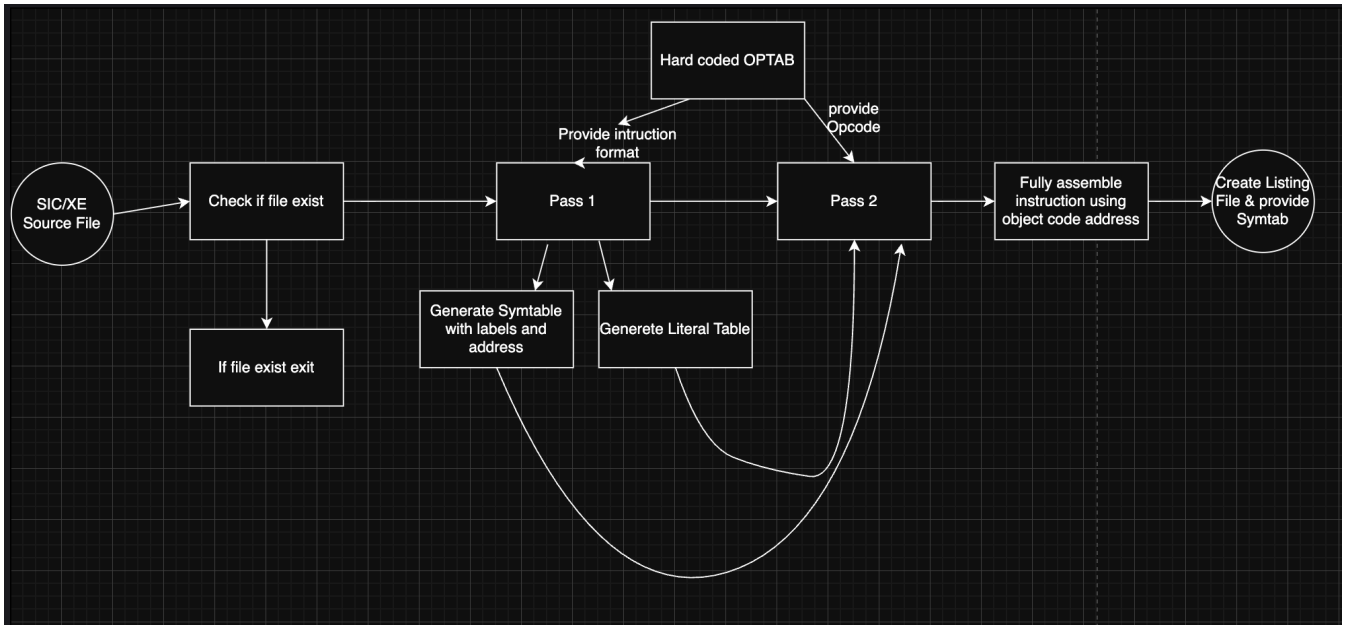
Functional Requirements:

- Program must accept SIC/XE source file via command line argument
- Must utilize a 2-pass assembler for converting .sic file to listing file
- Provide a hardcoded opcode table(OPTAB) used to determine instruction format and opcode
- Build Symtab in pass 1
- Build listing file in pass 2
- Final output must be complete listing file along with Symtab generated from the .sic file

System Requirements:

- Must compile and run on SDSU's EDORAS environment
- Must compile using g++
- Use Makefile in order to compile

System Design/Specification:



Development Environment:

- Integrated Developing Environment: VS Code and VIM
- Compiler: GCC Compiler
- Environment: SDSU EDORAS server running Oracle Linux Server 8.10
- Git

Run/Test Environment:

Using the samples provided on Canvas, we will run and test the environment first, we will run the run and test environment on the GCC compiler. We will do further test cases to ensure the program will successfully run. Finally, we will compile and test over on the Edoras server to ensure there are no errors.

Code and Test Plan:

1. Code Review:
    - Group members will review the source code to ensure no syntax errors are present.
    - Verify that all coding conventions are followed, and all requirements are included.
2. Unit Testing:
    - Checking correctness of Symtab
    - Checking LOCCTR increases correctly
    - Ensure instructions get properly converted to object code

3.) Integration Testing:

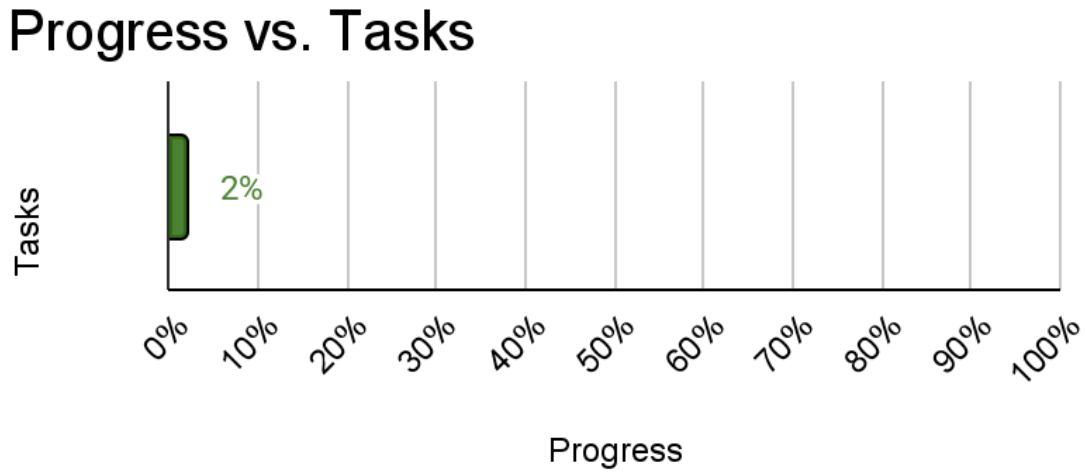
- Ensure program can accept multiple SIC/XE source files
- Check if symtab is properly passed onto from pass 1 to pass 2
- This is done to ensure that the entire program will work as intended.

Critical Path
System Design → Pass 1 Implementation → Pass 2 Implementation →
Output File Generation → Testing → Final Integration → Submission

POAM

ID	Description	Risk	Owners	Priority	Due date	Status
1	Develop pseudocode & System design for two-pass assembler	Glitches causing major rewrites	Manuel Jorge	High	03/10/26	In work
2	Implement pass 1 logic Build SYMTAB & track LOCCTR	Incorrect address pass 2 calculation could break	Jorge	High	03/25/26	Not started
3	Implement Pass 2 logic OPTAB & SYMTAB	Incorrect opcode lookup Or address handling	Manuel	High	03/25/26	Not started
4	Handle invalid input & Error detection	Program crash or freeze	Jorge	Med	04/09/26	Not started
5	Generate output files Listing file & symbol table	Incorrect formatting	Manuel	Med	04/09/26	Not started
6	Testing with sample SIC/XE programs	Incorrect formatting	Manuel	Med	04/09/26	Not started
7	Documentation SDD, comments, MakeFile	Bugs files	Jorge Manuel	High	04/09/26	Not started
8	Test pass1/pass 2	Compile errors	Jorge Manuel	High	04/09/26	Not started
9	Final integration	EDORAS issues	Jorge Manuel	High	04/09/26	Not started
10	Submit project	Packing or submission issues	Jorge Manuel	High	04/21/26	Not started

MileStone


Progress vs. Tasks
Tasks
2%
0%
10%
20%
30%
40%
50%
60%
70%
80%
90%
100%
Progress